

lamet-agent

完整技术文档

Agentic Workflow for Large-Momentum Effective Theory Analysis

2026 年 7 月 20 日

项目地址: <https://github.com/Greyyy-HJC/lamet-agent.git>

许可证: MIT

Python 版本要求: 3.10+

目录

1	项目概述	3
1.1	核心设计理念	3
1.2	五阶段分析工作流	3
2	安装指南	3
2.1	环境要求	3
2.2	安装步骤	3
3	清单 (Manifest) 配置	4
3.1	顶层结构	4
3.2	Metadata 字段	5
3.3	Inputs 配置	5
3.4	动量标签格式	6
3.5	标准 HDF5 关联函数格式	6
4	命令行接口	6
4.1	CLI 命令	6
4.2	run 命令参数	7
4.3	运行示例	7
5	项目结构	8
6	核心模块详解	9
6.1	代理工作流 (agent.py)	9
6.2	LLM 后端 (core/llm.py)	10
6.3	数据类型 (core/data.py)	10
6.4	工具系统 (core/tools.py)	11
6.5	清单系统 (manifest.py)	11
6.6	阶段参数合约 (manifest_params.py)	12
7	各阶段详解	12
7.1	关联函数分析 (correlator_analysis)	12
7.2	重整化 (renormalization)	12
7.3	傅里叶变换 (fourier_transform)	13
7.4	微扰匹配 (perturbative_matching)	13
7.5	外推 (extrapolation)	13

8 中间数据格式 (NetCDF)	14
8.1 产物链	14
8.2 Python 读写	14
8.3 外部工具读取	14
9 示例清单详解	15
9.1 pion_pdf_cg_manifest.json ——Pion PDF (Coulomb 规范)	15
9.2 pion_da_gi_manifest.json ——Pion DA (规范不变)	15
9.3 kaon_da_gi_manifest.json ——Kaon DA (规范不变)	15
9.4 sample_manifest.jsonc ——带注释参考清单	15
10 规划模式 (Plan Mode)	15
11 接口清单 (API Reference)	16
11.1 CLI 入口 (cli.py)	16
11.2 代理核心 (agent.py)	16
11.3 数据核心 (core/data.py)	16
11.4 LLM 会话 (core/llm.py)	17
11.5 阶段映射 (core/stages.py)	17
11.6 关联函数阶段工具 (stages/correlator/functions.py)	17
11.7 重整化阶段工具 (stages/renorm/functions.py)	18
11.8 傅里叶阶段工具 (stages/fourier/functions.py)	18
11.9 匹配阶段工具 (stages/matching/functions.py)	19
11.10 外推阶段工具 (stages/extrapolation/functions.py)	19
11.11 核函数 (kernels.py)	19
12 依赖关系	19
13 报告系统	19

1 项目概述

lamet-agent 是一个 Python AI 代理框架，专为大动量有效理论（LaMET）格点 QCD 分析设计。它将完整的 LaMET 分析工作流程自动化，从关联函数数据出发，经过重整化、傅里叶变换、微扰匹配和外推，最终产生物理分布函数（PDF、DA、TMD）。

1.1 核心理念

- **清单驱动 (Manifest-Driven)**: 通过 JSON 清单定义全局数据源池和分阶段任务列表
- **DAG 任务图**: 任务 ID 形成有向无环图，关联函数任务分组原始数据集，后续任务通过角色命名的输入消费上游任务
- **NetCDF 中间数据**: 阶段产物以 EnsembleData NetCDF 文件存储，支持自描述、可追踪的中间结果
- **LLM 驱动执行**: 每个阶段的任务由 LLM 代理自主调用工具完成

1.2 五阶段分析 workflow

1. correlator_analysis	→	stages/correlator/	(关联函数分析)
2. renormalization	→	stages/renorm/	(重整化)
3. fourier_transform	→	stages/fourier/	(傅里叶变换)
4. perturbative_matching	→	stages/matching/	(微扰匹配)
5. extrapolation	→	stages/extrapolation/	(外推)

重整化阶段支持三种方案：

- **比值方案 (ratio)**: 目标与分母在完整坐标网格上逐点相除
- **混合比值方案 (hybrid-ratio)**: 短距离比值 + 长距离指数修正
- **混合自重整化方案 (hybrid-self-renormalization)**: 全范围自重整化 + 短距离 $\overline{\text{MS}}$ 匹配

2 安装指南

2.1 环境要求

2.2 安装步骤

组件	要求
Python	≥ 3.10
核心依赖	
numpy	≥ 2.0
pydantic	≥ 2.8
typer	≥ 0.12
tqdm	≥ 4.68
分析依赖 (analysis extra)	
gvar	≥ 12.1
lsqfit	≥ 13.0
scipy	≥ 1.13
h5py	≥ 3.11
matplotlib	≥ 3.8
xarray	≥ 2024.1
netCDF4	≥ 1.7

表 1: 系统需求

```
1 # 1. 克隆仓库
2 git clone https://github.com/Greyyy-HJC/lamet-agent.git
3 cd lamet-agent
4
5 # 2. 创建虚拟环境
6 python3 -m venv .venv
7 source .venv/bin/activate
8
9 # 3. 安装核心包 + 开发和分析依赖
10 pip install -e ".[dev,analysis]"
11
12 # 4. (可选) 安装 Codex 后端支持
13 pip install -e ".[codex]"
```

3 清单 (Manifest) 配置

清单是 lamet-agent 的核心配置机制，定义了分析运行的所有参数。

3.1 顶层结构

```

1 {
2   "metadata": { ... },    // 运行级设置
3   "inputs":   { ... },    // 数据源（关联函数、外部产物、核函数）
4   "stages":   { ... }     // 阶段配置（默认值 + 任务列表）
5 }

```

3.2 Metadata 字段

字段	类型	说明
run_id	string	运行标识符
root_directory	string	根目录（所有相对路径的基准）
artifacts_directory	string	产物输出目录
target_observable	string	目标可观测量: "pdf" 或 "da"
resample_mode	string	重采样模式: "jk", "bs", 或 "gvar"
random_seed	int	随机种子（必需）
sample_error_mode	string	误差模式（默认 "covariance"）
bin_size	int	配置合并大小（可选）
workers	int	并行工作进程数（默认 1）
stages	list	有序的阶段 ID 列表

表 2: Metadata 参数

3.3 Inputs 配置

```

1 {
2   "inputs": {
3     "correlators": [
4       {
5         "correlator_id": "ensemble_2pt",
6         "correlator_type": "2pt",
7         "data_path": "data/ensemble_2pt.h5",
8         "ensemble": "HISQa060_X",
9         "hadron": "pion",
10        "gfix": "CG",
11        "source_operator": "g5",
12        "sink_operator": "g5",
13        "volume": "S48T64",
14        "lattice_spacing_fm": 0.0574,

```

```

15     "momentum": ["PX0PY0PZ0", "PX5PY0PZ0"]
16   }
17 ],
18 "artifacts": [
19   {
20     "id": "bare_pdf_reference",
21     "stage": "correlator_analysis",
22     "path": "artifacts/correlator_analysis/ref.nc"
23   }
24 ],
25 "kernels": [
26   {
27     "stage": "renormalization",
28     "kernel_id": "ZMSbar_pdf",
29     "kernel_path": "src/lamet_agent/kernels.py",
30     "scheme": "hybrid_ratio",
31     "kernel_parameters": { "mu": 2.0 }
32   }
33 ]
34 }
35 }

```

3.4 动量标签格式

动量标签采用精确格式 $PX_nPY_nPZ_n$ (每个分量为有符号整数), 物理动量由下式计算:

$$p [\text{GeV}] = \frac{2\pi\hbar c}{L_s a [\text{fm}]} \sqrt{n_x^2 + n_y^2 + n_z^2}, \quad \hbar c = 0.1973269804 \text{ GeV fm}$$

3.5 标准 HDF5 关联函数格式

- 普通 2pt: <source>/<sink>/<momentum>, 形状 (Lt, n_cfg)
- qDA 2pt: <source>/<sink>/<momentum>/bT<bT>/bz<bz>, 形状 (Lt, n_cfg)
- 3pt:<source>/<sink>/<current>/<momentum>/tsep<tsep>/bT<bT>/bz<bz>, 形状 (tsep+1, n_cfg)

4 命令行接口

4.1 CLI 命令

lamet-agent 通过 pyproject.toml 注册为命令行工具:

```
1 # 验证清单
2 lamet-agent validate examples/pion_pdf_cg_manifest.json
3
4 # 运行分析
5 lamet-agent run examples/pion_pdf_cg_manifest.json
6
7 # 交互式规划清单
8 lamet-agent plan draft_manifest.jsonc --backend api --model deepseek/deepseek-chat
```

4.2 run 命令参数

参数	说明
--backend	LLM 后端: mock, external, api, codex
--model	模型 ID (api: provider/model_id, codex: CODEX_MODEL_ID)
--verbose / -v	启用 ReAct 风格详细输出
--actions-path	external 后端的 JSONL 转录路径
--api-key-file	API 密钥文件路径 (默认 api.key)
--base-url	覆盖 API HTTP 端点
--report_language	报告语言: en 或 ch

表 3: run 命令参数

4.3 运行示例

```
1 # 使用 DeepSeek API 后端运行
2 lamet-agent run examples/pion_pdf_cg_manifest.json \
3     --backend api --model deepseek/deepseek-chat --verbose
4
5 # 使用 OpenAI 后端运行
6 lamet-agent run examples/pion_pdf_cg_manifest.json \
7     --backend api --model openai/gpt-4o
8
9 # 使用 Codex 后端运行
10 lamet-agent run examples/pion_pdf_cg_manifest.json \
11     --backend codex --model CODEX_MODEL_ID --verbose
12
13 # Mock 模式 (开发/测试)
14 lamet-agent run examples/pion_pdf_cg_manifest.json --backend mock
15
```

```

16 # 重放 JSONL 转录
17 lamet-agent run examples/pion_pdf_cg_manifest.json \
18     --backend external --actions-path actions.jsonl

```

5 项目结构

```

1 lamet-agent/
2 |— pyproject.toml          # 项目配置、依赖、入口点
3 |— README.md              # 完整文档
4 |— PLAN.md / AGENTS.md / PROJECT_LOG.md # 开发规划与日志
5 |— examples/              # 示例清单
6 |   |— sample_manifest.jsonc # 带注释的参考清单 (JSONC)
7 |   |— pion_pdf_cg_manifest.json # P0/P5 CG pion PDF
8 |   |— pion_pdf_gi_manifest.json # P0/P4 GI pion PDF
9 |   |— pion_da_gi_manifest.json # GI pion DA
10 |   |— kaon_da_gi_manifest.json # GI kaon DA
11 |   |— partial_sample_manifest.jsonc
12 |   └— fake_data/generate_fake_data.py
13 |— src/lamet_agent/       # 核心包
14 |   |— __init__.py
15 |   |— agent.py           # run_agent() 主循环
16 |   |— cli.py             # CLI 入口 (validate, run, plan)
17 |   |— manifest.py        # 清单模式与验证
18 |   |— manifest_params.py # 阶段参数合约注册表
19 |   |— kernels.py         # 内置核函数示例
20 |   |— core/              # 核心模块
21 |       |— data.py        # EnsembleData/EnsembleInfo 类型
22 |       |— llm.py         # 可插拔 LLM 会话后端
23 |       |— prompting.py   # 提示词构建
24 |       |— tools.py       # 工具注册、参数准备、绘图路径
25 |       |— stages.py      # 阶段 ID → 包映射
26 |       |— trace.py       # ReAct 风格追踪
27 |       |— banner.py      # 启动横幅
28 |       |— plotting.py    # 出版风格绑图
29 |       |— resampling.py  # Jackknife/Bootstrap 重采样
30 |       └— reporting.py   # 阶段报告
31 |   |— planning/          # 清单规划模式
32 |       |— agent.py       # 规划代理
33 |       |— core.py        # 规划核心逻辑
34 |       |— conversion.py  # HDF5 数据转换
35 |       |— questions.py   # 交互式问题
36 |       └— render.py      # 终端渲染
37 |   └— stages/            # 阶段实现

```

```

38 |         └─ correlator/          # 关联函数分析
39 |         └─ renorm/             # 重整化
40 |         └─ fourier/            # 傅里叶变换
41 |         └─ matching/           # 微扰匹配
42 |         └─ extrapolation/      # 外推
43 |         └─ review/             # 审查
44 |         (每个阶段含 prompts.py, skills.py, functions.py, reporting.py)
45 └─ tests/unit/                  # 单元测试

```

6 核心模块详解

6.1 代理 workflow (agent.py)

```

1 def run_agent(manifest_path, *, backend, model=None,
2               verbose=False, actions_path=None,
3               api_key_file=None, base_url=None,
4               report_language="en") -> dict:
5     """
6     执行完整的清单驱动分析工作流。
7
8     返回: {"run_id": ..., "status": ..., "summary": ...,
9           "stage_results": ..., "actions": ...}
10    """

```

执行流程:

1. CLI 接收清单路径和运行时选项
2. manifest.py 验证源 ID、任务 ID、有序依赖和路径
3. agent.py 按序执行 metadata.stages 列表
4. 对每个阶段任务:
 - (a) core/tools.validate_stage_inputs() 检查缺失输入
 - (b) core/prompting.build_stage_static_prompt() 组装静态上下文
 - (c) core/llm.make_llm_session() 创建 LLM 会话
 - (d) 多轮循环 (最多 40 个工具步骤): 模型发出 JSON 动作, 工具执行并返回观察
 - (e) 终端工具将主要数据放入 store["output"]
5. 阶段完成后生成报告
6. 输出紧凑的 JSON 摘要到 stdout

6.2 LLM 后端 (core/llm.py)

支持四种可插拔会话后端：

后端	说明	用途
mock	确定性脚手架	开发/测试
external	JSONL 转录重放	回归测试
api	OpenAI 兼容 HTTP API	生产运行
codex	Codex Python SDK	Codex 集成

表 4: LLM 后端类型

```

1 class LlmSession:
2     """可插拔 LLM 会话抽象"""
3
4 def make_llm_session(backend, *, model=None, api_key_file=None,
5                     base_url=None, actions_path=None) -> LlmSession:
6     """工厂函数：根据后端类型创建会话"""
7
8 def parse_api_model(spec: str) -> tuple[str, str]:
9     """解析 provider/model_id CLI 规格"""
10
11 # 预配置的 API 提供商
12 PROVIDERS = {
13     "deepseek": {"base_url": "https://api.deepseek.com/v1",
14                  "default_model": "deepseek-chat",
15                  "api_key_env": "DEEPSEEK_API_KEY"},
16     "openai": {"base_url": "https://api.openai.com/v1",
17                "default_model": "gpt-4o",
18                "api_key_env": "OPENAI_API_KEY"},
19 }
```

6.3 数据类型 (core/data.py)

```

1 class EnsembleInfo:
2     """系综元数据容器"""
3     ensemble: str
4     hadron: str
5     gfix: str
6     momentum: str
7     volume: str
8     lattice_spacing_fm: float
```

```
9     # ... 更多字段
10
11 class EnsembleData:
12     """重采样格点数据容器 (NetCDF 序列化) """
13     values: np.ndarray          # shape (n_sample, *physical_dims)
14     coords: dict                # 物理坐标
15     ensemble: EnsembleInfo      # 系综信息
16     resample: str               # 重采样模式
17
18     def to_netcdf(self, path: str) -> None
19     @classmethod
20     def from_netcdf(cls, path: str) -> EnsembleData
```

NetCDF 数据结构:

- 第一维度始终命名为 resample
- 复数数组原生往返 (auto_complex=True)
- 属性: ensemble (JSON 序列化的 EnsembleInfo)、resample

6.4 工具系统 (core/tools.py)

```
1 def prepare_tool_args(stage_name, tool_name, tool_args, manifest) -> dict:
2     """规范化 LLM 工具调用参数 (路径、保存路径等) """
3
4 def filter_tool_kwargs(tool_func, tool_args, *, strict=True) -> dict:
5     """过滤工具参数以匹配函数签名"""
6
7 def resolve_stage_tools(stage_name, job_store) -> dict:
8     """解析阶段的工具注册表"""
9
10 def resolve_plot_save_path(manifest, stage_name, job_id, stem) -> str:
11     """解析绘图保存路径到阶段产物目录"""
12
13 def validate_stage_inputs(stage_name, job, job_store,
14                           manifest_inputs) -> dict:
15     """验证阶段输入并返回 input_issues"""
```

6.5 清单系统 (manifest.py)

```
1 def load_manifest(path: str) -> dict:
2     """加载和解析清单文件 (JSON 或 JSONC) """
3
4 def validate_manifest(manifest: dict) -> list[str]:
```

```

5     """验证清单模式、任务 ID、有序依赖和路径"""
6
7 def resolve_manifest_paths(manifest: dict, manifest_dir: str) -> dict:
8     """解析所有相对路径"""

```

6.6 阶段参数合约 (manifest_params.py)

```

1 # 中央参数合约注册表
2 STAGE_PARAM_CONTRACTS = {
3     "correlator_analysis": { ... },
4     "renormalization":     { ... },
5     "fourier_transform":   { ... },
6     "perturbative_matching": { ... },
7     "extrapolation":       { ... },
8 }
9
10 def validate_stage_params(stage_name, defaults, job_params) -> list[str]:
11     """递归拒绝未知键 (defaults 和 params) """

```

7 各阶段详解

7.1 关联函数分析 (correlator_analysis)

工具集:

- inspect_correlator_scale: 选择关联函数缩放因子
- tune_ground_state: 2pt 窗口扫描 + 模型平均
- tune_bare_matrix: 在样本平均数据上扫描裸矩阵元拟合窗口
- fit_bare_matrix_grid: 将调优后的窗口应用于所有 z 和重采样样本

参数:

- fit_scope: 分析族 (3pt_ratio, FH, 3pt_ratio+FH, qda_ratio)
- model_average: 是否对拟合函数候选进行 BMA 组合
- fit_strategy: joint, chained, 或 independent

7.2 重整化 (renormalization)

方案:

- **ratio**: $h_s^R(z) = h_s^{\text{target}}(z)/h_s^{\text{denominator}}(z)$
- **hybrid_ratio**: 短距离比值 + 长距离指数修正
- **hybrid_self_renormalization**: 全范围自重整化 + $\overline{\text{MS}}$ 匹配

混合自重整化 workflow:

$$g(z) - \ln Z_{\overline{\text{MS}}}^{\text{PDF}}(z; \mu) \simeq m_0 z + b, \quad z_R(z, a) = \exp[\ln M_{\text{fit}}(z, a) - g(z) + m_0 z]$$

$$H_{\text{ren}}(z) = \frac{H_{\text{bare}}(z)}{z_R(z, a) Z_{\overline{\text{MS}}}(z)}$$

参数:

- **normalization**: 是否在重整化前除以 $z = 0$ 值
- **zs_fm**: 混合开关距离 (fm)
- **scheme_parameters.LambdaQCD_gev**: Λ_{QCD} (GeV)
- **scheme_parameters.d**: 连续/离散化系数
- **scheme_parameters.m0_gev**: m_0 参数
- **mu**: 重整化标度 (GeV)

7.3 傅里叶变换 (fourier_transform)

将坐标空间的矩阵元变换为准 PDF:

$$\tilde{g}(x, P_z) = \int \frac{dz}{2\pi} e^{ixzP_z} h^R(z, P_z)$$

7.4 微扰匹配 (perturbative_matching)

将准 PDF 匹配到光锥 PDF:

$$g(x, \mu) = \int \frac{dy}{|y|} C\left(\frac{x}{y}, \frac{\mu}{P_z}\right) \tilde{g}(y, P_z)$$

7.5 外推 (extrapolation)

将有限动量下的结果外推到无限动量极限。

阶段	示例产物
correlator_analysis	correlator_analysis/ca_p5.nc
renormalization	renormalization/rn_p5.nc
fourier_transform	fourier_results/fourier_result.nc
perturbative_matching	matching_results/quasi_pdf.nc

表 5: 典型产物链

8 中间数据格式 (NetCDF)

8.1 产物链

8.2 Python 读写

```

1 from lamet_agent.core.data import EnsembleData
2
3 # 写入
4 data.to_netcdf("artifacts/fourier_results/fourier_result.nc")
5
6 # 读取
7 reload = EnsembleData.from_netcdf("artifacts/fourier_results/fourier_result.nc")

```

8.3 外部工具读取

```

1 # 使用 ncdump 查看
2 ncdump -h artifacts/fourier_results/fourier_result.nc

```

```

1 # 使用 xarray 读取 (无需 lamet-agent)
2 import json, xarray as xr
3 from lamet_agent.core.data import EnsembleInfo
4
5 da = xr.load_dataarray("fourier_result.nc", auto_complex=True)
6 ensemble = EnsembleInfo(**json.loads(da.attrs["ensemble"]))
7 values = da.values # shape (n_sample, *physical_dims)

```

9 示例清单详解

9.1 pion_pdf_cg_manifest.json ——Pion PDF (Coulomb 规范)

完整的 P0/P5 工作流，包含关联函数分析、混合比值重整化、傅里叶变换和微扰匹配。

9.2 pion_da_gi_manifest.json ——Pion DA (规范不变)

完整的 GI pion DA 工作流，从 qDA 关联函数分析到匹配和审查。

9.3 kaon_da_gi_manifest.json ——Kaon DA (规范不变)

完整的 GI kaon DA 工作流，运行辅助脚本位于 runs/ds_kaon_da_gi/。

9.4 sample_manifest.jsonc ——带注释参考清单

标记为 JSONC (JSON with comments)，每个字段都带有文档注释。

10 规划模式 (Plan Mode)

lamet-agent plan 命令接受不完整的 JSON/JSONC 清单，运行 LLM 控制的规划循环：

```
1 lamet-agent plan draft_manifest.jsonc --backend api --model deepseek/deepseek-chat
```

规划循环功能：

- 检查清单完整性
- 检查 HDF5 输入
- 提出终端问题
- 应用经验证的 JSON Patch 编辑
- 生成快速清单 (jackknife + 均值误差) 和完整清单 (协方差 + BMA)
- 自动转换非标准 HDF5 布局

终端摘要组织为： 缺失参数、不一致设置、建议修改、数据转换。

11 接口清单 (API Reference)

11.1 CLI 入口 (cli.py)

```
1 def entrypoint():
2     """CLI 入口点: 注册 validate, run, plan 命令"""
3
4 def cmd_validate(manifest_path: str):
5     """验证清单模式与核函数导入"""
6
7 def cmd_run(manifest_path, *, backend, model, verbose,
8             actions_path, api_key_file, base_url, report_language):
9     """运行完整的阶段循环并收集结构化动作"""
10
11 def cmd_plan(draft_path, *, backend, model, api_key_file, base_url):
12     """交互式规划清单"""
```

11.2 代理核心 (agent.py)

```
1 def run_agent(manifest_path, *, backend, model=None,
2               verbose=False, actions_path=None,
3               api_key_file=None, base_url=None,
4               report_language="en") -> dict:
5     """
6     执行 metadata.stages, 按序运行每个声明的任务,
7     使用隔离的 store, 将 store["output"] 注册到任务 ID 下。
8     """
```

11.3 数据核心 (core/data.py)

```
1 @dataclass
2 class EnsembleInfo:
3     """系综元数据"""
4     ensemble: str
5     hadron: str = ""
6     gfix: str = ""
7     momentum: str = ""
8     volume: str = ""
9     lattice_spacing_fm: float = 0.0
10    bz_direction: str = ""
11    # ... 更多字段
12
13 @dataclass
```

```

14 class EnsembleData:
15     """重采样格点数据"""
16     values: np.ndarray
17     coords: dict[str, np.ndarray]
18     ensemble: EnsembleInfo
19     resample: str = "jk"
20     attrs: dict = field(default_factory=dict)
21
22     def to_netcdf(self, path: str) -> None: ...
23     @classmethod
24     def from_netcdf(cls, path: str) -> EnsembleData: ...

```

11.4 LLM 会话 (core/llm.py)

```

1 class LlmSession(ABC):
2     @abstractmethod
3     def send(self, messages: list[dict]) -> dict: ...
4
5 class MockSession(LlmSession): ...
6 class ExternalSession(LlmSession): ...
7 class ApiSession(LlmSession): ...
8 class CodexSession(LlmSession): ...
9
10 def make_llm_session(backend, **kwargs) -> LlmSession: ...
11 def parse_api_model(spec: str) -> tuple[str, str]: ...

```

11.5 阶段映射 (core/stages.py)

```

1 STAGE_PACKAGES = {
2     "correlator_analysis": "lamet_agent.stages.correlator",
3     "renormalization": "lamet_agent.stages.renorm",
4     "fourier_transform": "lamet_agent.stages.fourier",
5     "perturbative_matching": "lamet_agent.stages.matching",
6     "extrapolation": "lamet_agent.stages.extrapolation",
7 }

```

11.6 关联函数阶段工具 (stages/correlator/functions.py)

```

1 def inspect_correlator_scale(...) -> dict:
2     """选择关联函数缩放因子"""
3
4 def tune_ground_state(...) -> dict:

```

```
5     """2pt 窗口扫描 + 模型平均"""
6
7 def tune_bare_matrix(...) -> dict:
8     """在样本平均数据上扫描裸矩阵元拟合窗口"""
9
10 def fit_bare_matrix_grid(...) -> dict:
11     """将统一调优的窗口应用于所有 z 和重采样样本"""
12
13 STAGE_TOOLS = {
14     "inspect_correlator_scale": inspect_correlator_scale,
15     "tune_ground_state": tune_ground_state,
16     "tune_bare_matrix": tune_bare_matrix,
17     "fit_bare_matrix_grid": fit_bare_matrix_grid,
18 }
```

11.7 重整化阶段工具 (stages/renorm/functions.py)

```
1 def apply_ratio_scheme_renormalization(...) -> dict:
2     """应用比值方案重整化"""
3
4 def apply_hybrid_ratio_renormalization(...) -> dict:
5     """应用混合比值方案"""
6
7 def fit_self_renormalization_factor(...) -> dict:
8     """拟合自重整化因子"""
9
10 def apply_self_renormalization(...) -> dict:
11     """应用自重整化因子"""
12
13 def plot_self_renormalization_diagnostics(...) -> dict:
14     """绑制自重整化诊断图"""
15
16 def plot_renormalized_matrix_element(...) -> dict:
17     """绑制重整化矩阵元图"""
```

11.8 傅里叶阶段工具 (stages/fourier/functions.py)

```
1 def fourier_transform(...) -> dict:
2     """执行傅里叶变换 (坐标空间 → 动量空间) """
3
4 def plot_fourier_result(...) -> dict:
5     """绑制傅里叶变换结果"""
```

11.9 匹配阶段工具 (stages/matching/functions.py)

```
1 def apply_matching(...) -> dict:
2     """应用微扰匹配核函数"""
3
4 def plot_matching_result(...) -> dict:
5     """绑制匹配结果"""
```

11.10 外推阶段工具 (stages/extrapolation/functions.py)

```
1 def extrapolate(...) -> dict:
2     """执行无限动量外推"""
3
4 def plot_extrapolation(...) -> dict:
5     """绑制外推结果"""
```

11.11 核函数 (kernels.py)

```
1 # 内置核函数示例，用于烟雾测试
2 # 生产核函数通过清单中的 kernel_path 指定
```

12 依赖关系

13 报告系统

每个阶段完成后，`reporting.py` 模块自动生成指定语言 (en/ch) 的分析报告，包含：

- 阶段执行摘要
- 工具调用日志
- 拟合参数与结果
- 绑图输出路径
- 中间数据产物路径

依赖	版本	用途
Python	≥ 3.10	运行环境
numpy	≥ 2.0	数值计算
pydantic	≥ 2.8	数据验证与模式
typer	≥ 0.12	CLI 框架
tqdm	≥ 4.68	进度条
gvar	≥ 12.1	误差传播 (分析 extra)
lsqfit	≥ 13.0	最小二乘拟合 (分析 extra)
scipy	≥ 1.13	科学计算 (分析 extra)
h5py	≥ 3.11	HDF5 I/O (分析 extra)
matplotlib	≥ 3.8	绑图 (分析 extra)
xarray	≥ 2024.1	标注数组 (分析 extra)
netCDF4	≥ 1.7	NetCDF I/O (分析 extra)
openai-codex	$\geq 0.1.0b3$	Codex 后端 (codex extra)

表 6: 完整依赖列表